

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №5
По дисциплине: «Интеллектуальный анализ данных»
Тема: «Деревья решений»

Выполнила:
Студент 4 курса
Группы ИИ-26
Карпович К.А.
Проверил:
Никитин С.К.

Брест 2026

Цель работы: на практике сравнить работу нескольких алгоритмов одиночного дерева решений, случайного леса и бустинга для деревьев решений.

Задачи:

1. Загрузить датасет по варианту;
2. Разделить данные на обучающую и тестовую выборки;
3. Обучить на обучающей выборке одиночное дерево, случайный лес и реализовать бустинг для решающих деревьев (AdaBoost, CatBoost, XGBoost);
4. Оценить точность каждой модели на тестовой выборке;
5. Сравнить результаты, сделать выводы о применимости каждого метода для данного набора данных.

Вариант 6

- Pima Indians Diabetes
- Предсказать наличие диабета у пациенток
- **Задания:**
 1. Загрузите данные и стандартизируйте их;
 2. Разделите выборку в соотношении 70/30;
 3. Обучить на обучающей выборке одиночное дерево, случайный лес и реализовать бустинг для решающих деревьев (AdaBoost, CatBoost, XGBoost);
 4. Сравните производительность моделей с помощью метрики recall для положительного класса (наличие диабета).
 5. Обоснуйте, какой из алгоритмов предпочтительнее для данной медицинской задачи.

Описание данных

Датасет Pima Indians Diabetes содержит данные о 768 пациентках и 8 признаках. Целевая переменная Outcome равна 1, если у пациентки есть диабет, и 0, если нет.

Признаки: Pregnancies (число беременностей), Glucose (глюкоза), BloodPressure (артериальное давление), SkinThickness (толщина кожной складки), Insulin (инсулин), BMI (индекс массы тела), DiabetesPedigreeFunction (наследственная предрасположенность), Age (возраст).

Классы несбалансированы: 500 пациенток (65.1%) без диабета и 268 (34.9%) с диабетом.

Таблица 1 – Распределение классов

Класс	Количество	Доля
0 (нет диабета)	500	65.1%
1 (диабет)	268	34.9%

В пяти признаках встречаются нулевые значения, физически невозможные для живого человека, то есть это пропуски.

Таблица 2 – Количество нулевых значений

Признак	Количество нулей
Glucose	5
BloodPressure	35
SkinThickness	227
Insulin	374
BMI	11

Предобработка данных

1. Нулевые значения в столбцах Glucose, BloodPressure, SkinThickness, Insulin и BMI были признаны пропусками и заменены медианой соответствующего признака. Медиана рассчитывалась по обучающей выборке.
2. Выборка разделена на обучающую и тестовую в соотношении 70/30 со стратификацией по целевому признаку (random_state=42). Обучающая выборка содержит 537 записей (доля класса 1 – 34.8%), тестовая – 231 запись (доля класса 1 – 35.1%).
3. Выполнена стандартизация признаков (StandardScaler). Параметры стандартизации рассчитаны только по обучающей выборке и затем применены к тестовой, чтобы избежать утечки данных. После стандартизации среднее значение каждого признака в обучающей выборке равно 0, стандартное отклонение равно 1.

Стандартизация и разделение выполнены в таком порядке, чтобы информация о тестовой выборке не попала в обучение. Для деревьев масштаб признаков не влияет на качество, но стандартизация выполнена по условию задания.

Используемые модели

- **Одиночное дерево решений** – базовая модель. Рассмотрены два варианта: без ограничений глубины и с ограничением max_depth=4.
- **Случайный лес** – ансамбль независимых деревьев, обученных на случайных подвыборках данных и признаков (бэггинг). Итоговый прогноз определяется голосованием деревьев.
- **AdaBoost** – последовательный бустинг: каждое следующее простое дерево уделяет больше внимания объектам, на которых ошиблись предыдущие.
- **CatBoost и XGBoost** – градиентный бустинг над деревьями, где каждое новое дерево исправляет ошибки текущего ансамбля.

Таблица 3 – Параметры моделей

Модель	Основные параметры
Дерево (без ограничений)	параметры по умолчанию
Дерево (max_depth=4)	max_depth=4, min_samples_leaf=5
Случайный лес	n_estimators=300, max_depth=6, min_samples_leaf=3
AdaBoost	n_estimators=200, learning_rate=0.5, базовая модель – дерево глубины 1
CatBoost	iterations=300, depth=4, learning_rate=0.05
XGBoost	n_estimators=300, max_depth=3, learning_rate=0.05, subsample=0.8, colsample_bytree=0.8

Для всех моделей использовался random_state=42. Параметры заданы заранее и не подбирались по тестовой выборке. Эксперимент проведён в двух вариантах: со стандартными весами классов и с учётом дисбаланса (class_weight='balanced' для деревьев и леса, auto_class_weights='Balanced' для CatBoost, scale_pos_weight ≈ 1.9 для XGBoost, то есть отношение числа здоровых к числу больных в обучающей выборке). У AdaBoost в используемой реализации веса классов не поддерживаются, поэтому его результаты в обоих вариантах совпадают.

Код программы:

```
import os
import warnings

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from catboost import CatBoostClassifier
from sklearn.ensemble import AdaBoostClassifier, RandomForestClassifier
from sklearn.impute import SimpleImputer
from sklearn.metrics import (
    accuracy_score,
```

```

confusion_matrix,
f1_score,
precision_score,
recall_score,
roc_auc_score,
)
from sklearn.model_selection import StratifiedKFold, cross_val_score, train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier
from xgboost import XGBClassifier

pd.set_option("display.max_columns", None)
pd.set_option("display.width", 250)

RANDOM_STATE = 42
LOCAL_FILE = "diabetes.csv"
URL = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.data.csv"
COLUMNS = [
    "Pregnancies",
    "Glucose",
    "BloodPressure",
    "SkinThickness",
    "Insulin",
    "BMI",
    "DiabetesPedigreeFunction",
    "Age",
    "Outcome",
]
ZERO_AS_MISSING = ["Glucose", "BloodPressure", "SkinThickness", "Insulin", "BMI"]

def load_data():
    if os.path.exists(LOCAL_FILE):
        df = pd.read_csv(LOCAL_FILE)
        if "Outcome" not in df.columns:
            df = pd.read_csv(LOCAL_FILE, header=None, names=COLUMNS)
    else:
        df = pd.read_csv(URL, header=None, names=COLUMNS)
    return df

df = load_data()

print("=" * 70)
print("1. ЗАГРУЗКА ДАННЫХ")
print("=" * 70)
print("Размер датасета:", df.shape)
print(df.head())
print("\nРаспределение классов:")
print(df["Outcome"].value_counts())
print(df["Outcome"].value_counts(normalize=True).round(3))
print("\nКоличество нулевых значений в признаках, где ноль невозможен:")
print((df[ZERO_AS_MISSING] == 0).sum())

df[ZERO_AS_MISSING] = df[ZERO_AS_MISSING].replace(0, np.nan)

X = df.drop(columns="Outcome")
y = df["Outcome"]
feature_names = X.columns.tolist()

print("\n" + "=" * 70)
print("2. РАЗДЕЛЕНИЕ 70/30 И СТАНДАРТИЗАЦИЯ")
print("=" * 70)
X_train_raw, X_test_raw, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=RANDOM_STATE, stratify=y
)

imputer = SimpleImputer(strategy="median")
scaler = StandardScaler()

X_train = scaler.fit_transform(imputer.fit_transform(X_train_raw))
X_test = scaler.transform(imputer.transform(X_test_raw))

print("Обучающая выборка:", X_train.shape, "| доля класса 1: %.3f" % y_train.mean())
print("Тестовая выборка: ", X_test.shape, "| доля класса 1: %.3f" % y_test.mean())
print("Среднее обучающей после стандартизации:", np.round(X_train.mean(axis=0), 3))
print("Std обучающей после стандартизации: ", np.round(X_train.std(axis=0), 3))

```

```

def build_models(balanced=False):
    neg = (y_train == 0).sum()
    pos = (y_train == 1).sum()
    cw = "balanced" if balanced else None
    spw = neg / pos if balanced else 1.0
    cb_w = "Balanced" if balanced else None

    return {
        "Одиночное дерево (без ограничений)": DecisionTreeClassifier(
            class_weight=cw, random_state=RANDOM_STATE
        ),
        "Одиночное дерево (max_depth=4)": DecisionTreeClassifier(
            max_depth=4, min_samples_leaf=5, class_weight=cw, random_state=RANDOM_STATE
        ),
        "Случайный лес": RandomForestClassifier(
            n_estimators=300,
            max_depth=6,
            min_samples_leaf=3,
            class_weight=cw,
            random_state=RANDOM_STATE,
            n_jobs=-1,
        ),
        "AdaBoost": AdaBoostClassifier(
            estimator=DecisionTreeClassifier(max_depth=1),
            n_estimators=200,
            learning_rate=0.5,
            random_state=RANDOM_STATE,
        ),
        "CatBoost": CatBoostClassifier(
            iterations=300,
            depth=4,
            learning_rate=0.05,
            auto_class_weights=cb_w,
            random_seed=RANDOM_STATE,
            verbose=0,
            allow_writing_files=False,
        ),
        "XGBoost": XGBClassifier(
            n_estimators=300,
            max_depth=3,
            learning_rate=0.05,
            subsample=0.8,
            colsample_bytree=0.8,
            scale_pos_weight=spw,
            eval_metric="logloss",
            random_state=RANDOM_STATE,
            n_jobs=-1,
        ),
    }

```

```

def evaluate(models, title):
    print("\n" + "=" * 70)
    print(title)
    print("=" * 70)

    rows = []
    fitted = {}
    cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)

    for name, model in models.items():
        model.fit(X_train, y_train)
        fitted[name] = model

        y_pred = model.predict(X_test)
        y_proba = model.predict_proba(X_test)[: , 1]
        tn, fp, fn, tp = confusion_matrix(y_test, y_pred).ravel()

        cv_recall = cross_val_score(
            model, X_train, y_train, cv=cv, scoring="recall", n_jobs=1
        )

        rows.append(
            {
                "Модель": name,
                "Recall (класс 1)": recall_score(y_test, y_pred, pos_label=1),
            }
        )

```

```

        "Precision": precision_score(y_test, y_pred, pos_label=1),
        "F1": f1_score(y_test, y_pred, pos_label=1),
        "Accuracy": accuracy_score(y_test, y_pred),
        "ROC-AUC": roc_auc_score(y_test, y_proba),
        "TP": tp,
        "FN": fn,
        "FP": fp,
        "TN": tn,
        "CV recall (train, 5 fold)": cv_recall.mean(),
    }
)

result = pd.DataFrame(rows).set_index("Модель")
result = result.sort_values("Recall (класс 1)", ascending=False)
pd.set_option("display.width", 220)
pd.set_option("display.max_columns", 30)
print(result.round(3))
return result, fitted

results_base, fitted_base = evaluate(
    build_models(balanced=False),
    "3-4. ОБУЧЕНИЕ МОДЕЛЕЙ И ОЦЕНКА НА ТЕСТЕ (стандартные веса классов)",
)

results_bal, fitted_bal = evaluate(
    build_models(balanced=True),
    "5. ТЕ ЖЕ МОДЕЛИ С УЧЁТОМ ДИСБАЛАНСА КЛАССОВ (class_weight / scale_pos_weight)",
)

summary = pd.DataFrame(
    {
        "Recall (стандартные веса)": results_base["Recall (класс 1)"],
        "Recall (balanced)": results_bal["Recall (класс 1)"],
        "Precision (balanced)": results_bal["Precision"],
        "ROC-AUC (balanced)": results_bal["ROC-AUC"],
    }
).sort_values("Recall (balanced)", ascending=False)

print("\n" + "=" * 70)
print("СВОДНАЯ ТАБЛИЦА")
print("=" * 70)
print(summary.round(3))

results_base.round(4).to_csv("results_standard.csv", encoding="utf-8-sig")
results_bal.round(4).to_csv("results_balanced.csv", encoding="utf-8-sig")
summary.round(4).to_csv("results_summary.csv", encoding="utf-8-sig")

order = results_base.index.tolist()
x = np.arange(len(order))
width = 0.38

fig, ax = plt.subplots(figsize=(11, 5.5))
b1 = ax.bar(
    x - width / 2,
    results_base.loc[order, "Recall (класс 1)"],
    width,
    label="Стандартные веса",
)
b2 = ax.bar(
    x + width / 2,
    results_bal.loc[order, "Recall (класс 1)"],
    width,
    label="С учётом дисбаланса",
)
for bars in (b1, b2):
    for bar in bars:
        ax.text(
            bar.get_x() + bar.get_width() / 2,
            bar.get_height() + 0.01,
            "%0.2f" % bar.get_height(),
            ha="center",
            fontsize=9,
        )
ax.set_xticks(x)
ax.set_xticklabels(order, rotation=20, ha="right")
ax.set_ylabel("Recall (класс 1 — диабет)")
ax.set_ylim(0, 1.05)

```

```

ax.set_title("Сравнение моделей по recall на тестовой выборке")
ax.legend()
ax.grid(axis="y", alpha=0.3)
plt.tight_layout()
plt.savefig("recall_comparison.png", dpi=150)
plt.show()

best_name = results_bal.index[0]
print("\nЛучшая модель по recall (balanced):", best_name)

fig, axes = plt.subplots(2, 3, figsize=(14, 8))
for ax, (name, model) in zip(axes.ravel(), fitted_bal.items()):
    cm = confusion_matrix(y_test, model.predict(X_test))
    ax.imshow(cm, cmap="Blues")
    ax.set_title(name, fontsize=10)
    ax.set_xticks([0, 1])
    ax.set_yticks([0, 1])
    ax.set_xticklabels(["Нет", "Диабет"])
    ax.set_yticklabels(["Нет", "Диабет"])
    ax.set_xlabel("Предсказано")
    ax.set_ylabel("Истина")
    for i in range(2):
        for j in range(2):
            ax.text(
                j,
                i,
                cm[i, j],
                ha="center",
                va="center",
                color="white" if cm[i, j] > cm.max() / 2 else "black",
                fontsize=13,
            )
plt.suptitle("Матрицы ошибок (модели с учётом дисбаланса классов)")
plt.tight_layout()
plt.savefig("confusion_matrices.png", dpi=150)
plt.show()

importances = pd.DataFrame(index=feature_names)
for name in ["Случайный лес", "AdaBoost", "CatBoost", "XGBoost"]:
    model = fitted_bal[name]
    imp = np.asarray(model.feature_importances_, dtype=float)
    importances[name] = imp / imp.sum()

print("\nВажность признаков:")
print(importances.round(3))

importances.plot(kind="bar", figsize=(12, 5.5))
plt.ylabel("Нормированная важность")
plt.title("Важность признаков в ансамблевых моделях")
plt.xticks(rotation=30, ha="right")
plt.grid(axis="y", alpha=0.3)
plt.tight_layout()
plt.savefig("feature_importance.png", dpi=150)
plt.show()

print("\nГотово. Файлы сохранены рядом со скриптом:")
print("results_standard.csv, results_balanced.csv, results_summary.csv")
print("recall_comparison.png, confusion_matrices.png, feature_importance.png")

```

Результат программы:

1. ЗАГРУЗКА ДАННЫХ

=====

Размер датасета: (768, 9)

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

Распределение классов:

Outcome

0 500

1 268

Name: count, dtype: int64

Outcome

0 0.651

1 0.349

Name: proportion, dtype: float64

Количество нулевых значений в признаках, где ноль невозможен:

Glucose 5

BloodPressure 35

SkinThickness 227

Insulin 374

BMI 11

dtype: int64

2. РАЗДЕЛЕНИЕ 70/30 И СТАНДАРТИЗАЦИЯ

=====

Обучающая выборка: (537, 8) | доля класса 1: 0.348

Тестовая выборка: (231, 8) | доля класса 1: 0.351

Среднее обучающей после стандартизации: [0. 0. -0. 0. 0. -0. 0. -0.]

Std обучающей после стандартизации: [1. 1. 1. 1. 1. 1. 1. 1.]

=====

3-4. ОБУЧЕНИЕ МОДЕЛЕЙ И ОЦЕНКА НА ТЕСТЕ (стандартные веса классов)

=====

	Recall (класс 1)	Precision	F1	Accuracy	ROC-AUC	TP	FN	FP	TN	CV recall (train, 5 fold)
Модель										
XGBoost	0.543	0.629	0.583	0.727	0.819	44	37	26	124	0.578
Одиночное дерево (без ограничений)	0.531	0.573	0.551	0.697	0.659	43	38	32	118	0.551
AdaBoost	0.531	0.705	0.606	0.758	0.829	43	38	18	132	0.562
Случайный лес	0.531	0.672	0.593	0.745	0.825	43	38	21	129	0.567
CatBoost	0.531	0.642	0.581	0.732	0.824	43	38	24	126	0.589
Одиночное дерево (max_depth=4)	0.383	0.633	0.477	0.706	0.757	31	50	18	132	0.465

=====

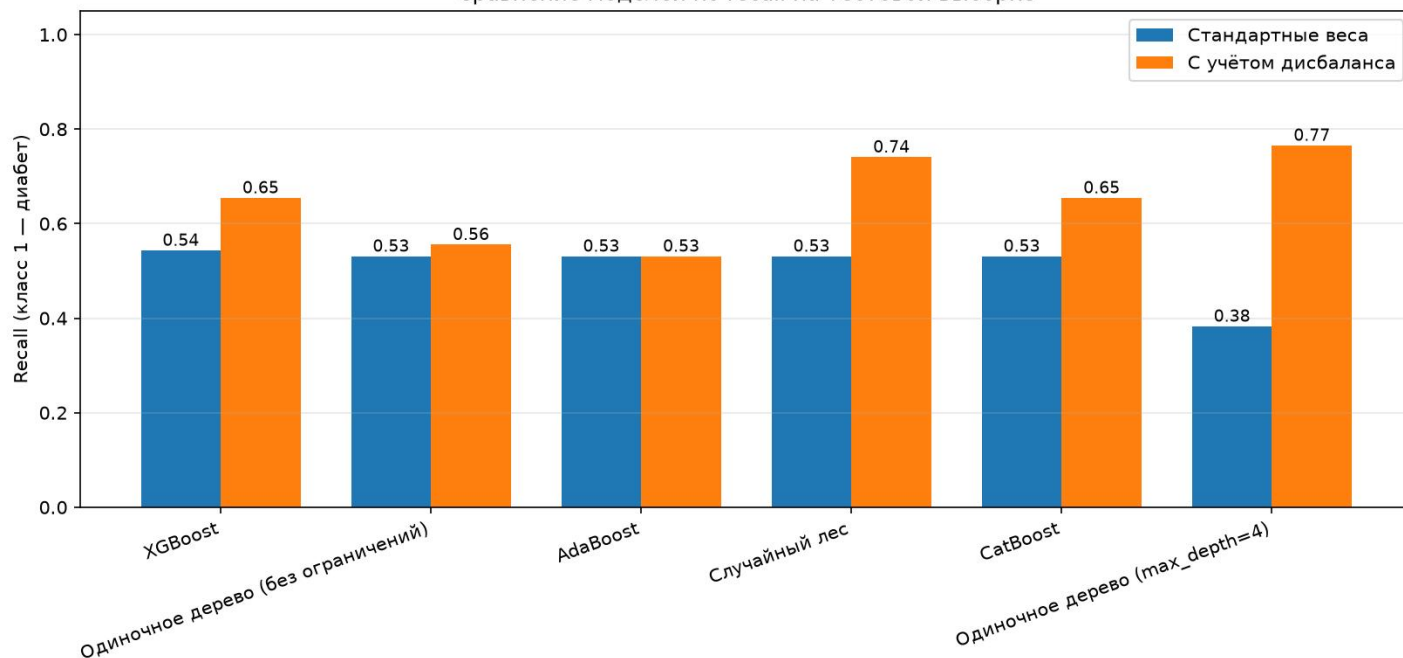
5. ТЕ ЖЕ МОДЕЛИ С УЧЁТОМ ДИСБАЛАНСА КЛАССОВ (class_weight / scale_pos_weight)

=====

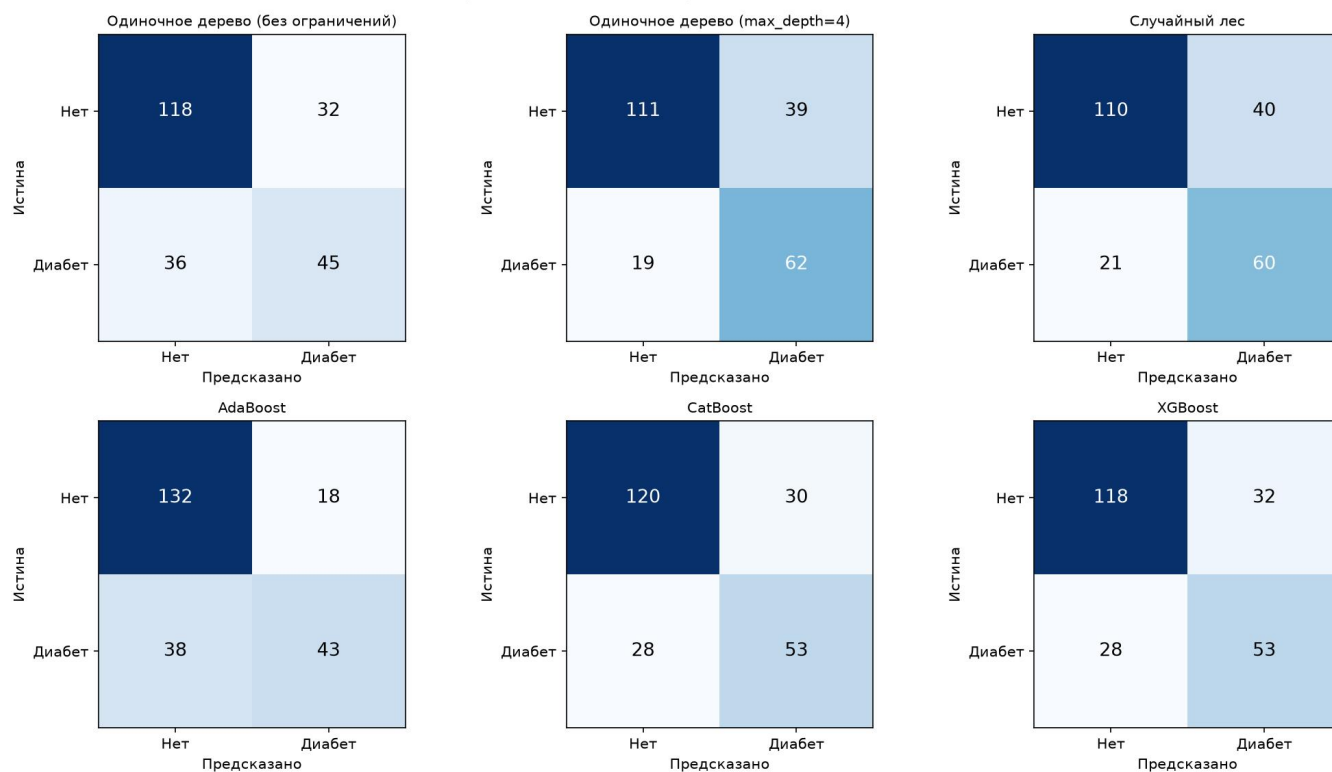
	Recall (класс 1)	Precision	F1	Accuracy	ROC-AUC	TP	FN	FP	TN	CV recall (train, 5 fold)
Модель										
Одиночное дерево (max_depth=4)	0.765	0.614	0.681	0.749	0.757	62	19	39	111	0.781
Случайный лес	0.741	0.600	0.663	0.736	0.838	60	21	40	110	0.728
XGBoost	0.654	0.624	0.639	0.740	0.821	53	28	32	118	0.658
CatBoost	0.654	0.639	0.646	0.749	0.828	53	28	30	120	0.674
Одиночное дерево (без ограничений)	0.556	0.584	0.570	0.706	0.671	45	36	32	118	0.492

AdaBoost	0.531	0.705	0.606	0.758	0.829	43	38	18	132	0.562
=====										
СВОДНАЯ ТАБЛИЦА										
=====										
	Recall (стандартные веса)		Recall (balanced)		Precision (balanced)		ROC-AUC (balanced)			
Модель										
Одиночное дерево (max_depth=4)	0.383		0.765		0.614		0.757			
Случайный лес	0.531		0.741		0.600		0.838			
XGBoost	0.543		0.654		0.624		0.821			
CatBoost	0.531		0.654		0.639		0.828			
Одиночное дерево (без ограничений)	0.531		0.556		0.584		0.671			
AdaBoost	0.531		0.531		0.705		0.829			
Лучшая модель по recall (balanced): Одиночное дерево (max_depth=4)										
Важность признаков:										
	Случайный лес	AdaBoost	CatBoost	XGBoost						
Pregnancies	0.069	0.043	0.067	0.107						
Glucose	0.322	0.416	0.259	0.252						
BloodPressure	0.051	0.008	0.084	0.082						
SkinThickness	0.055	0.000	0.061	0.083						
Insulin	0.103	0.024	0.101	0.113						
BMI	0.178	0.302	0.162	0.140						
DiabetesPedigreeFunction	0.093	0.140	0.139	0.093						
Age	0.128	0.066	0.126	0.129						

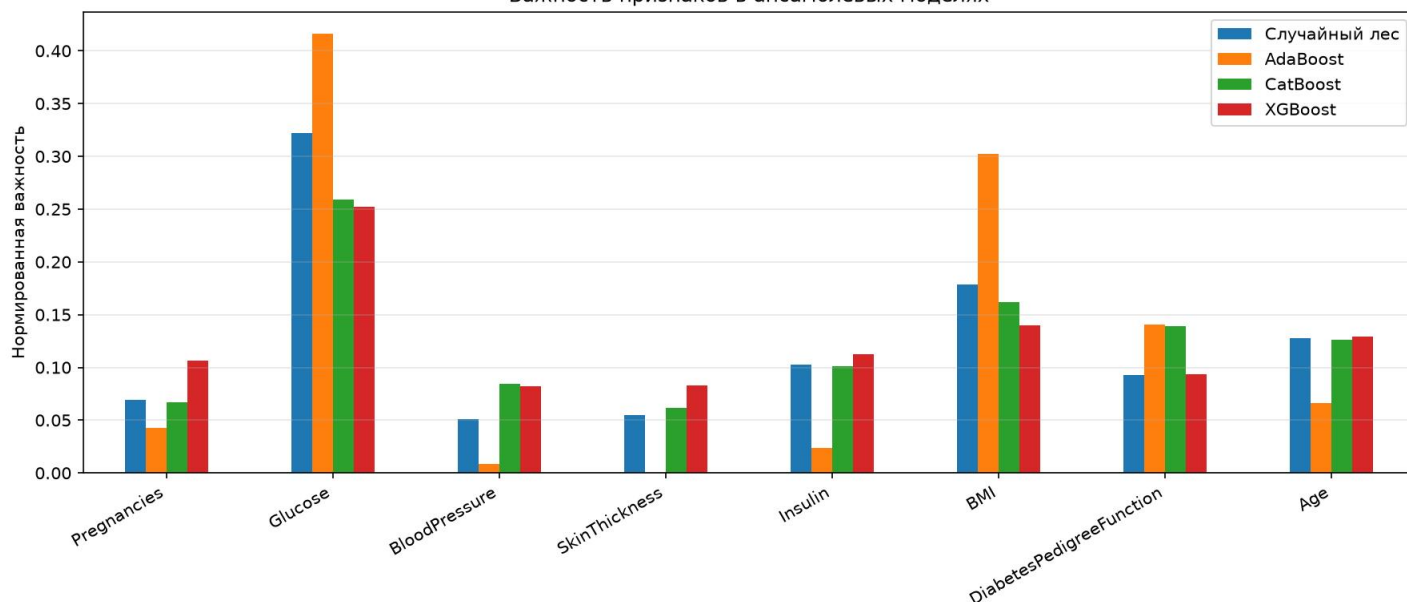
Сравнение моделей по recall на тестовой выборке



Матрицы ошибок (модели с учётом дисбаланса классов)



Важность признаков в ансамблевых моделях



Анализ результатов

- Стандартные настройки.** Пять из шести моделей показали практически одинаковый recall (0.53–0.54), то есть пропускали 37–38 из 81 больной пациентки. Отличия между ними по этой метрике незначительны. Одиночное дерево с `max_depth=4` оказалось заметно хуже (0.383). Высокая accuracy при низком recall показывает, что на несбалансированных данных одной accuracy доверять нельзя.
- Влияние учёта дисбаланса.** Веса классов заметно повысили recall: у дерева с `max_depth=4` с 0.383 до 0.765, у случайного леса с 0.531 до 0.741, у CatBoost и XGBoost до 0.654. У дерева без ограничений улучшение небольшое (0.531 → 0.556), а у AdaBoost recall не изменился, так как веса классов в нём не использовались.

3. **Цена роста recall.** Рост recall сопровождается ростом числа ложноположительных срабатываний: у дерева с `max_depth=4` FP выросло с 18 до 39, у случайного леса с 21 до 40. Precision при этом остаётся на уровне 0.60–0.64. Для скрининга это допустимо: ложная тревога приводит лишь к дообследованию, а пропущенный диагноз к позднему лечению.
4. **Переобучение одиночного дерева.** Дерево без ограничений глубины имеет самый низкий ROC-AUC (0.659–0.671), а его CV recall в режиме с весами классов снижается до 0.492. Это признак переобучения: дерево подстраивается под обучающие данные. Ограничение глубины и минимального размера листа улучшает обобщающую способность.
5. **Качество ранжирования.** По ROC-AUC лучшие результаты показывают ансамбли: случайный лес (0.838), AdaBoost (0.829), CatBoost (0.828), XGBoost (0.821). У дерева с `max_depth=4` AUC ниже (0.757).
6. **Важность признаков.** Во всех ансамблях наиболее значимым признаком является Glucose (0.25–0.42), затем идёт BMI (0.14–0.30). Это согласуется с медицинскими представлениями: уровень глюкозы и избыточная масса тела – ключевые факторы риска диабета. AdaBoost опирается в основном на два-три признака, а SkinThickness практически не использует. Важность Insulin и SkinThickness следует трактовать осторожно, так как значительная часть их значений (374 и 227 из 768) была восстановлена медианой.
7. **Согласованность оценок.** CV recall на обучающей выборке близок к recall на тесте (например, у случайного леса 0.728 и 0.741), поэтому результат нельзя считать случайным. Тем не менее тестовая выборка содержит всего 81 положительный пример, поэтому разница в 2–3 пациентки между моделями статистически незначима.

Выбор алгоритма для медицинской задачи

В задаче диагностики диабета наиболее опасна ошибка второго рода: больная пациентка признана здоровой и не получает лечения. Поэтому главным критерием выбран recall для класса «диабет», а precision, ROC-AUC и матрица ошибок позволяют убедиться, что высокий recall не достигнут за счёт того, что почти все пациентки объявлены больными.

Формально наибольший recall (0.765) показало одиночное дерево с `max_depth=4`, но его преимущество перед случайным лесом (0.741) составляет две пациентки из 81 (62 против 60 правильно найденных). При этом у случайного леса наибольший ROC-AUC (0.838 против 0.757 у дерева), и он устойчивее: ансамбль из 300 деревьев меньше зависит от особенностей конкретной обучающей выборки, чем единичное дерево, структура которого может заметно меняться при небольших изменениях данных. Поэтому **для данной медицинской задачи предпочтительным является случайный лес с учётом дисбаланса классов.**

Одиночное дерево с ограничением глубины целесообразно использовать там, где важна интерпретируемость: его правила можно показать врачу в виде короткой схемы. CatBoost и XGBoost при настройках по умолчанию дают более осторожные прогнозы (recall 0.654), но имеют резерв улучшения за счёт подбора гиперпараметров и снижения порога классификации. AdaBoost при близком ROC-AUC (0.829) не поддерживает веса классов и находит лишь 53% больных, поэтому для задачи, где важен recall, без доработки он малоприменим.

Вывод: на практике сравнила работу нескольких алгоритмов одиночного дерева решений, случайного леса и бустинга для деревьев решений.